

Proyecto 1: Entrega 3:

Repositorio

Marlon Gutiérrez Vásquez, C33619

Escuela de Ingeniería Eléctrica

Universidad de Costa Rica

IE0435 - Inteligencia Artificial Aplicada a la Ingeniería Eléctrica, I-2026, Grupo 01

Profesor: Marvin Coto Jiménez.

I. RESUMEN

En este proyecto se desarrolló un sistema de procesamiento digital de imágenes y clasificación supervisada para detectar la presencia de contaminación por granos de arroz en un conjunto de muestras. El procedimiento inició con la toma de imágenes RGB, las cuales fueron procesadas mediante un programa de preprocesamiento digital de imágenes. Dicho programa convierte las imágenes en imágenes segmentadas, aplica un proceso de umbralización para obtener imágenes binarias, genera imágenes reconstruidas a partir de los datos almacenados para verificar que el CSV se construyó de forma correcta y finalmente crea archivos CSV con cada imagen representada como un vector fila.

El procedimiento se aplicó tanto al conjunto de entrenamiento como al conjunto de prueba. El conjunto de entrenamiento contiene 120 imágenes, mientras que el conjunto de prueba contiene 30 imágenes independientes.

Posteriormente, se utilizó un segundo programa encargado del entrenamiento y evaluación de diferentes modelos de clasificación supervisada. A partir de las imágenes almacenadas en los archivos CSV, el programa extrae 17 características geométricas, adaptativas y poblacionales por imagen.

Los modelos evaluados fueron Árbol de Decisión, Naive Bayes, SVM con kernel RBF para diferentes valores de C, SVM lineal y KNN. A partir de los resultados obtenidos, el mejor modelo fue SVM RBF con $C=1$, el cual alcanzó un Accuracy de 0.9000 y un F1 Score de 0.9032.

Palabras clave: procesamiento digital de imágenes, segmentación, CSV, clasificación supervisada, SVM RBF, StandardScaler, Accuracy, Precision, Recall, F1 Score.

II. ESTRUCTURA GENERAL DEL PROYECTO

La organización final del proyecto se preparó para ser subida a un repositorio Git. Para ello se utilizaron cuatro directorios principales, los cuales permiten separar adecuadamente los datos de entrenamiento, los datos de prueba, los programas desarrollados y el modelo final entrenado.

- **conjunto_entrenamiento:** contiene todos los archivos relacionados con el conjunto de datos utilizado durante la etapa de entrenamiento de los modelos de clasificación supervisada. Dentro de este directorio se almacenan las 120 imágenes utilizadas para entrenar los algoritmos,

así como sus respectivas imágenes binarias, imágenes reconstruidas y archivos CSV generados durante el pre-procesamiento.

- **conjunto_prueba:** contiene el conjunto independiente de 30 imágenes utilizado para evaluar el desempeño de los modelos entrenados. Este directorio posee la misma estructura interna que el conjunto de entrenamiento, permitiendo almacenar las imágenes RGB originales, las imágenes binarias segmentadas, las imágenes reconstruidas y el archivo CSV correspondiente al conjunto de prueba.
- **mejor_modelo:** contiene el modelo final seleccionado después de realizar la comparación entre los diferentes algoritmos de clasificación. En este directorio se almacena el archivo `C33619_Marlon_Gutierrez.joblib`, el cual incluye el modelo entrenado.
- **src:** contiene los programas fuente desarrollados en Python para el proyecto. Dentro de este directorio se encuentran los archivos `preprocesamiento_digital_imagenes.py` y `entrenamiento_y_prueba.py`, encargados del procesamiento digital de imágenes, generación de archivos CSV, reconstrucción de imágenes, extracción de características, entrenamiento de modelos y evaluación de resultados.

Los directorios **conjunto_entrenamiento** y **conjunto_prueba** poseen además la siguiente estructura interna:

- **CSV:** almacena el archivo CSV generado a partir de las imágenes segmentadas. Cada fila del archivo representa una imagen convertida en vector fila junto con su etiqueta correspondiente.
- **imagenes_RGB:** contiene las imágenes originales utilizadas como entrada del sistema de procesamiento digital de imágenes.
- **imagenes_binarias:** contiene las imágenes segmentadas generadas después de aplicar el proceso de umbralización y separación entre fondo y objetos.
- **imagenes_reconstruidas:** contiene las imágenes reconstruidas a partir de los datos almacenados dentro del archivo CSV. Estas imágenes permiten verificar visualmente que la información fue almacenada correctamente.

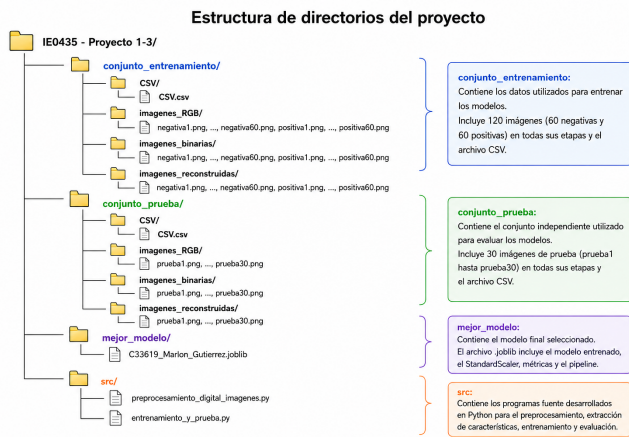


Figura 1: Estructura general del proyecto.

III. DESCRIPCIÓN DEL PROCEDIMIENTO LÓGICO PARA EL PROCESAMIENTO DE DATOS.

En esta sección se describe el procedimiento tomado para generar el conjunto de datos que posteriormente se usarán para el entrenamiento de los distintos modelos.

Para ejemplificar el procedimiento se utilizará la imagen **negativa1** (ver Figura. 2) del conjunto de entrenamiento. Sin embargo, es importante recalcar que este mismo proceso se aplicó para todas las muestras del conjunto de entrenamiento y de prueba.

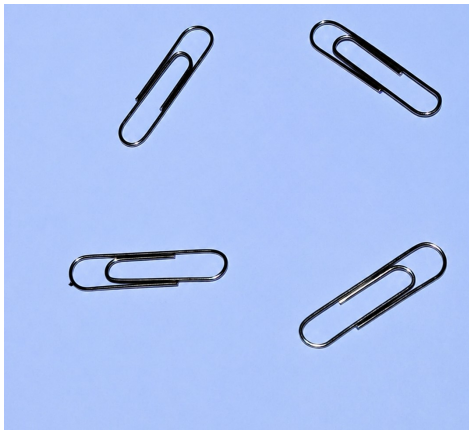


Figura 2: Imagen RGB negativa1 del conjunto de entrenamiento

III-A. Generación del conjunto de entrenamiento

En esta sección se explica todo el proceso tomado para poder generar el conjunto de datos que posteriormente serán utilizados para entrenar y probar el desempeño de los modelos.

III-A1. toma de imágenes : El primer paso del procedimiento consistió en tomar las imágenes RGB de las muestras utilizadas en el proyecto. En total se trabajó con 30 imágenes, de las cuales 15 corresponden a muestras positivas y 15 a muestras negativas. Estas imágenes fueron nombradas siguiendo una numeración lógica, por ejemplo, positiva1, positiva2,

hasta positiva15, y de igual manera negativa1, negativa2, hasta negativa15.

Las imágenes fueron tomadas en un día soleado y al aire libre, con el propósito de reducir problemas asociados a una iluminación deficiente o muy variable. Se realizó de esta manera ya que, en procesamiento digital de imágenes, los cambios fuertes de iluminación pueden afectar los valores de intensidad de los píxeles y, por lo tanto, pueden modificar el resultado de las imágenes segmentadas.

Una vez tomadas las imágenes, estas fueron almacenadas en el directorio `imagenes_RGB` del conjunto de entrenamiento.

III-A2. Ampliación del conjunto de datos: Aunque inicialmente se contaba con un total de 30 imágenes para el desarrollo del proyecto, la meta establecida consistía en construir un conjunto de entrenamiento más robusto de 120 imágenes, con el fin de mejorar el proceso de entrenamiento y evaluación de los distintos modelos de clasificación implementados.

Para lograr esta ampliación del conjunto de datos, se incorporaron imágenes adicionales aportadas por los compañeros Walther Barrantes, Ignacio Montenegro, Sebastián Rojas. Estas nuevas muestras fueron tomadas siguiendo condiciones similares a las utilizadas originalmente, procurando mantener fondos claros, buena iluminación y un adecuado contraste entre los objetos y el fondo, con el propósito de conservar uniformidad dentro del conjunto de datos.

Las imágenes agregadas se enumeraron continuando la secuencia previamente definida. De esta manera, las imágenes negativas adicionales fueron nombradas desde negativa16 hasta negativa60, mientras que las imágenes positivas fueron nombradas desde positiva16 hasta positiva60. Con ello, se logró completar un total de 120 imágenes dentro del conjunto de entrenamiento, distribuidas equitativamente entre muestras positivas y negativas. Posteriormente, todas las imágenes incorporadas fueron almacenadas dentro del directorio `imagenes_RGB` del conjunto de entrenamiento.

III-B. Generación del conjunto de prueba

Con el propósito de evaluar el desempeño final de los modelos entrenados, se construyó un conjunto de prueba independiente al conjunto de entrenamiento. Para ello, se utilizaron 30 imágenes RGB adjuntadas por el estudiante Jorge Loría.

Estas imágenes fueron almacenadas dentro del directorio `imagenes_RGB` correspondiente al conjunto de prueba. Además, se nombraron siguiendo una secuencia lógica bajo los nombres prueba1, hasta prueba30.

Es importante aclarar que a todas las imágenes de este conjunto de prueba se les aplicó el mismo preprocedimiento digital de imágenes que se va a explicar a continuación, con la salvedad de que los todos los resultados se van a guardar dentro de los subcarpetas del directorio `conjunto_prueba`, como se explicó anteriormente para ejemplificar el proceso solo se va a mostrar los resultados de la imagen negativa1 del conjunto de entrenamiento, pero este proceso se aplica por igual para todas las imágenes, tanto del conjunto de entrenamiento como en el de prueba.

III-B1. Instalación de OpenCV: Para realizar el procesamiento digital de imágenes se utilizó la biblioteca OpenCV. Esta biblioteca es una herramienta ampliamente usada en visión por computador, ya que permite trabajar con imágenes digitales, leer archivos de imagen, modificar sus canales de color, aplicar transformaciones, realizar segmentación y desarrollar algoritmos de análisis visual.

En este proyecto, OpenCV fue necesario principalmente para cargar las imágenes, manipular sus valores de píxeles y apoyar el proceso de segmentación. Para instalar esta biblioteca en Python, se utilizó el siguiente comando desde la terminal:

```
pip install opencv-python
```

Con esta instalación se habilitó el uso de funciones de procesamiento de imágenes dentro de los programas desarrollados en el proyecto.

III-B2. Programa para el preprocesamiento digital de imágenes: El programa principal desarrollado para esta entrega se llamó `preprocesamiento_digital_de_imagenes.py` y fue almacenado en el directorio `src`. Este programa se encarga de realizar el procesamiento completo de las imágenes originales hasta obtener las imágenes segmentadas y los datos necesarios para generar el archivo CSV.

El primer paso que realiza el programa es cargar las imágenes RGB almacenadas en el directorio `imagenes_RGB` tanto del conjunto de entrenamiento como al de prueba. Cada imagen RGB se encuentra compuesta por tres canales de color: rojo, verde y azul, conocidos como componentes R, G y B. Esto significa que cada píxel de la imagen original se representa mediante tres valores numéricos.

Apartir de las componentes R, G y B el programa calcula la componente de intensidad de cada imagen. Esto permite pasar de una representación basada en tres componentes de color a una representación basada en un único valor de intensidad por píxel. En otras palabras, el color de cada píxel deja de estar representado por tres valores R, G y B, y pasa a estar representado por un solo valor que indica qué tan claro u oscuro es dicho píxel.

Después de obtener la imagen de intensidad, se aplica un proceso de segmentación mediante el algoritmo de Kittler. Este algoritmo permite determinar un valor de umbral a partir del histograma de intensidades de la imagen. La idea general consiste en separar los píxeles de la imagen en dos grupos: uno asociado al fondo y otro asociado al objeto de interés. El algoritmo busca un umbral que minimice el error de clasificación entre ambos grupos, de manera que la separación entre fondo y objeto sea lo más adecuada posible.

La elección de este algoritmo se relaciona con los conocimientos adquiridos en la optativa de *Visión por Computador*, donde se estudiaron diferentes algoritmos de segmentación de imágenes, específicamente los métodos de Otsu, Kurita y Kittler. De los cuales, el de Kittler fue el que mejores resultados generó.

Es importante mencionar que el valor del umbral no se definió de forma manual para todas las imágenes, ya que cada imagen puede presentar pequeñas variaciones en la intensidad luminosa, sombras, reflejos o diferencias propias de la muestra. Por este motivo, puede resultar más conveniente utilizar un nivel de umbral diferente para cada imagen. Por lo tanto, el programa calcula el valor del umbral de forma independiente para cada una de las 120 imágenes del conjunto de entrenamiento y para las 30 del conjunto de prueba.

III-B3. Generación e inspección de las imágenes segmentadas: Una vez aplicado el algoritmo de segmentación, las imágenes resultantes se guardaron en el directorio `imagenes_binarias` de cada respectivo directorio para el conjunto de entrenamiento y de prueba. En estos directorios se pueden observar las imágenes binarias obtenidas después del procesamiento. En estas imágenes, el fondo y el objeto de interés quedan separados mediante valores de píxel distintos, donde el fondo quedó de color blanco y los objetos de color negro, lo que permite identificar visualmente si la segmentación fue realizada correctamente. Además, se redimensionaron las imágenes para tener un tamaño de 128x128 píxeles. En la Figura 3 se puede ver la representación segmentada de la imagen negativa.

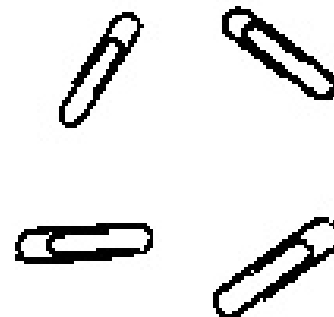


Figura 3: Imagen negativa segmentada.

Para esta entrega se inspeccionó cada imagen de forma individual, con el objetivo de garantizar que la segmentación se hubiera realizado adecuadamente. Esta revisión fue importante porque permitió detectar posibles errores antes de generar el archivo CSV para cada conjunto de datos. En particular, se observó que algunas imágenes negativas podían presentar pequeños puntos negros en zonas donde no deberían aparecer. Estos puntos podrían interpretarse como ruido dentro de la imagen y provocar que, posteriormente, un algoritmo de clasificación los confundiera con características propias de una muestra positiva.

Por esta razón, la inspección visual de las imágenes segmentadas fue una etapa necesaria del procedimiento. Esta revisión permitió comprobar que las imágenes no presentaran ruido significativo y que la separación entre fondo y muestra fuera adecuada antes de continuar con el almacenamiento de los datos.

III-B4. Generación del archivo CSV: Una vez que las imágenes se encontraban correctamente segmentadas, se procedió con la generación de los archivos CSV. El archivo generado se llamó `CSV.csv` y fue guardado en el directorio CSV correspondiente para cada conjunto de datos.

Para generar los archivos, cada imagen segmentada se interpretó como una matriz de píxeles. Debido a que las imágenes utilizadas tienen dimensiones de 128x128 píxeles, cada imagen puede representarse como una matriz de 128 filas y 128 columnas. Sin embargo, para almacenar esta información dentro de un archivo CSV, cada matriz fue convertida en un vector fila. Es decir, los valores de la matriz se reorganizaron en una sola fila, de manera que cada fila del archivo CSV representa una imagen segmentada completa. En la Figura 4 se puede ver la estructura de los CSVs.

Al final de cada fila se colocó la etiqueta correspondiente solicitada en el enunciado del proyecto. Esta etiqueta permite identificar la clase de cada imagen: se colocó un valor de 1 para las imágenes positivas y un valor de 0 para las imágenes negativas.

[illegible]

Figura 4: Archivo CSV.csv del conjunto de entrenamiento visualizado en Excel.

Cabe recalcar que, dentro del archivo `CSV.csv` del conjunto de entrenamiento, los primeros 60 elementos corresponden a las muestras positivas, siguiendo el orden lógico de numeración desde `positiva1` hasta `positiva60`. Por su parte, los últimos 60 elementos corresponden a las muestras negativas, desde `negativa1` hasta `negativa60`. Y a como lo solicitaba en el enunciado, en la última columna se colocó la etiqueta dependiendo si la muestra era positiva (etiqueta 1) y negativa (etiqueta 0)

III-B5. Verificación de la correcta estructura del archivo CSV: Analizar directamente el archivo CSV resulta complejo, debido a que contiene una gran cantidad de columnas. Cada imagen segmentada de 128x128 píxeles genera 16384 valores de píxel, a los cuales se les agrega la etiqueta correspondiente. Por esta razón, no es sencillo revisar visualmente en Excel si los datos ingresados representan correctamente las imágenes segmentadas.

Además, Excel no siempre permite visualizar o manipular cómodamente archivos tan grandes,

especialmente cuando contienen una cantidad elevada de columnas. Por estos motivos, se expandió el programa `preprocesamiento_digital_imagenes.py`. De tal forma que también se encarga de leer todos los elementos almacenados en el archivo `CSV.csv` de cada conjunto de datos y a partir de cada fila, el programa toma los valores correspondientes a los píxeles, los reorganiza nuevamente en una matriz de 128x128 píxeles y reconstruye la imagen segmentada original. Luego, estas imágenes reconstruidas se guardan en el directorio `imagenes_reconstruidas` de cada conjunto de datos respectivamente, para poder compararlas visualmente con las imágenes segmentadas generadas inicialmente.

Además, el programa lee el último dato de cada fila, correspondiente a la columna de etiquetas. Esto permite verificar si cada imagen reconstruida mantiene la etiqueta correcta, es decir, 1 para las muestras positivas y 0 para las muestras negativas.

En esta etapa se compararon todas las imágenes segmentadas con sus respectivas imágenes reconstruidas de forma individual. Esta comparación permitió comprobar visualmente que los datos almacenados en los archivos CSV eran correctos y que cada fila del archivo representaba adecuadamente a la imagen correspondiente. Para ejemplificar esto en la Figura 5 (a) se muestra la imagen negativa segmentada, mientras que la Figura 5 (b) se aprecia la imagen negativa reconstruida, se puede apreciar que ambas imágenes son exactamente igual, de esta forma se puede verificar que el contenido de los archivos CSV.csv es correcto.



(a) Imagen egativa1 segmenta-
da. (b) Imagen negati-
va1 reconstruida.

Figura 5: Comparación entre la imagen segmentada original y la imagen reconstruida de la muestra negativa1.

IV. ENTRENAMIENTO Y EVALUACIÓN DE LOS MODELOS DE CLASIFICACIÓN

En esta sección se describe el procedimiento utilizado para entrenar, evaluar y comparar los distintos modelos de clasificación implementados en el proyecto. Para ello, se desarrolló el programa `entrenamiento_y_prueba.py`, el cual se encuentra almacenado dentro del directorio `src` del proyecto.

Este programa se encarga automáticamente de realizar todas las etapas necesarias para el entrenamiento y prueba de los modelos de clasificación a continuación se describe cada una de ellas

IV-A. Carga y preprocesamiento de los datos

El primer paso realizado por el programa consiste en cargar los archivos CSV generados durante el procesamiento digital de imágenes. Ya que estos archivos contienen la representación vectorial de cada imagen segmentada junto con su respectiva etiqueta de clasificación.

IV-B. Extracción de características

Con el propósito de mejorar el desempeño de los modelos de clasificación, el programa implementa un proceso de extracción de características a partir de las imágenes binarias segmentadas [3], [5].

En lugar de utilizar directamente todos los píxeles de la imagen como entrada para los algoritmos, el programa calcula un conjunto de 17 características representativas para cada muestra. Estas características describen propiedades geométricas, poblacionales y morfológicas de los objetos presentes en la imagen, lo cual resulta apropiado para este proyecto debido a que los granos de arroz pueden diferenciarse mediante propiedades como tamaño, forma, elongación, compacidad y solidez [4]. Las cuales son

- **Número de objetos detectados:** cantidad total de regiones identificadas en la imagen segmentada.
- **Área media:** promedio del área de los objetos detectados.
- **Desviación estándar del área:** variación del tamaño de los objetos detectados.
- **Relación de aspecto media:** promedio de la relación entre el lado mayor y el lado menor de cada objeto.
- **Desviación estándar de la relación de aspecto:** variación en la forma alargada de los objetos.
- **Solidez media:** promedio de la relación entre el área del objeto y el área de su envolvente convexa [5].
- **Desviación estándar de la solidez:** variación de la solidez entre los objetos detectados.
- **Elongación media:** promedio del grado de alargamiento de los objetos.
- **Desviación estándar de la elongación:** variación del alargamiento entre los objetos.
- **Compacidad media:** medida que relaciona el área y el perímetro para describir qué tan compacto es cada objeto.
- **Número de objetos candidatos a arroz:** cantidad de objetos que cumplen con los criterios de área, aspecto y solidez.
- **Fracción de arroz:** proporción entre los objetos candidatos a arroz y el total de objetos detectados.
- **Coefficiente de variación del área:** relación entre la desviación estándar y la media del área.
- **Coefficiente de variación del aspecto:** relación entre la desviación estándar y la media de la relación de aspecto.
- **Número de objetos separados por watershed:** cantidad adicional de objetos detectados después de separar regiones superpuestas.
- **Puntaje poblacional:** proporción de objetos con áreas similares respecto a la mediana.

- **Entropía del área:** métrica que describe la diversidad de tamaños presentes en la imagen.

Al utilizar este conjunto de parámetros se logra establecer un conjunto de características permite que los modelos aprendan propiedades físicas y geométricas reales de los granos de arroz, en lugar de depender únicamente de patrones de intensidad de píxeles. Esto puede mejorar significativamente la capacidad de generalización de los modelos y reducir problemas asociados al ruido o variaciones leves de iluminación [3].

IV-C. Separación de objetos superpuestos

Durante el proceso de segmentación, puede ocurrir que varios granos de arroz aparezcan unidos o superpuestos dentro de una misma región binaria. En estos casos, múltiples objetos pueden interpretarse erróneamente como un único elemento, afectando negativamente las características calculadas y, por consiguiente, el desempeño de los modelos de clasificación [6].

Para reducir este problema, el programa implementa la técnica de segmentación *watershed*. Este método utiliza la transformada de distancia para estimar los centros de cada objeto presente en la imagen y posteriormente generar una separación entre regiones conectadas [7].

Gracias a este procedimiento, es posible identificar de mejor manera los distintos granos de arroz presentes en una misma agrupación, permitiendo obtener características geométricas más precisas y representativas de cada muestra analizada.

IV-D. Modelos de clasificación evaluados

Una vez extraídas las características, se procedió al entrenamiento y evaluación de distintos modelos de clasificación supervisada [8].

Los modelos evaluados dentro del programa fueron los siguientes:

- **Árbol de decisión:** modelo que clasifica los datos mediante una estructura de decisiones sucesivas. En cada etapa se evalúa una característica y se divide el conjunto de datos hasta llegar a una clase final [9].
- **Naive Bayes Gaussiano:** modelo probabilístico que calcula la probabilidad de que una muestra pertenezca a una clase. Se asume que las características siguen una distribución gaussiana y que son independientes entre sí [10].
- **SVM con kernel RBF:** modelo que busca separar las clases mediante una frontera de decisión no lineal. En este caso se evaluaron tres variantes con $C = 1$, $C = 10$ y $C = 100$. El parámetro C controla qué tanto se penalizan los errores de clasificación durante el entrenamiento [11].
- **SVM lineal:** modelo similar al SVM anterior, pero utiliza una frontera de decisión lineal. Se usa cuando las clases pueden separarse aproximadamente mediante una línea o hiperplano [11].
- **KNN:** modelo que clasifica una muestra según la clase de sus vecinos más cercanos. En este caso se evaluaron

dos variantes con $k = 3$ y $k = 5$, donde k indica cuántos vecinos se toman en cuenta para decidir la clase [12].

Cada uno de estos modelos fue entrenado utilizando el conjunto de entrenamiento y posteriormente evaluado utilizando el conjunto de prueba, permitiendo comparar su capacidad de clasificación sobre datos no utilizados durante el entrenamiento.

IV-E. Métricas de evaluación

Para evaluar el desempeño de los modelos de clasificación se utilizaron distintas métricas ampliamente empleadas en problemas de clasificación binaria [13]. Las métricas utilizadas se describen a continuación:

La métrica *Accuracy* representa el porcentaje total de clasificaciones correctas realizadas por el modelo y se calcula mediante:

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN}$$

La métrica *Precision* indica qué proporción de las predicciones positivas realizadas por el modelo fueron realmente correctas:

$$Precision = \frac{VP}{VP + FP}$$

La métrica *Recall* representa la capacidad del modelo para detectar correctamente las muestras positivas:

$$Recall = \frac{VP}{VP + FN}$$

Finalmente, el *F1 Score* corresponde a la media armónica entre *Precision* y *Recall* [14]:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Estas métricas permitieron comparar objetivamente el desempeño de cada modelo implementado.

IV-F. Resultados

Una vez ejecutado el programa `entrenamiento_y_prueba.py`, este imprime automáticamente en la terminal una tabla comparativa con el desempeño obtenido por cada uno de los modelos evaluados.

En la Figura 6 se muestra salida generada por el programa durante el proceso de evaluación de modelos.

TABLA COMPARATIVA				
	Accuracy	Precision	Recall	F1
Arbol de Decision	0.5333	0.5294	0.6000	0.5625
Naive Bayes	0.8000	1.0000	0.6000	0.7500
SVM RBF C1	0.9000	0.8750	0.9333	0.9032
SVM RBF C10	0.8333	0.7778	0.9333	0.8485
SVM RBF C100	0.5667	0.5455	0.8000	0.6486
SVM Lineal	0.8333	0.8125	0.8667	0.8387
KNN k=3	0.7667	0.7000	0.9333	0.8000
KNN k=5	0.8667	0.8235	0.9333	0.8750

Figura 6: Resultados de la evaluación de los modelos de clasificación.

IV-G. Elección del mejor modelo

A partir de los resultados obtenidos, se determinó que el modelo con mejor desempeño fue el SVM con kernel RBF y parámetro $C = 1$, obteniendo las siguientes métricas:

- Accuracy = 0.9000
- Precision = 0.8750
- Recall = 0.9333
- F1 Score = 0.9032

La principal razón de esta elección fue que este modelo obtuvo el mayor valor de *F1 Score*, alcanzando un valor de 0.9032. Esto indica que el clasificador logró un excelente equilibrio entre *Precision* y *Recall*, manteniendo una alta capacidad de detección de muestras positivas sin incrementar excesivamente la cantidad de falsas clasificaciones.

Asimismo, este modelo presentó una mejor capacidad de generalización sobre el conjunto de prueba, logrando una menor cantidad de errores en comparación con los demás clasificadores evaluados.

Dado que este es el mejor modelo, se salva este algoritmo de clasificación bajo el nombre `C33619_Marlon_Gutierrez.joblib` en el directorio `mejor_modelo`.

V. CONCLUSIONES

- A partir del desarrollo de este proyecto se logró implementar un procedimiento completo de procesamiento digital de imágenes en Python, desde la lectura de imágenes RGB hasta la generación y verificación de un archivo CSV con los datos segmentados. Esto permitió comprender de forma práctica cómo una imagen puede transformarse en una matriz de datos numéricos y cómo dicha información puede almacenarse para su uso posterior.
- La toma de imágenes en condiciones de iluminación similares permitió reducir variaciones importantes entre las muestras. Sin embargo, se comprobó que no es recomendable utilizar un único umbral fijo para todas las imágenes, ya que cada una puede presentar pequeñas diferencias de intensidad. Por esta razón, el uso del algoritmo de Kittler resultó conveniente, debido a que permite calcular un umbral independiente para cada imagen a partir de su distribución de intensidades.
- También se concluye que la inspección visual de las imágenes segmentadas es una etapa importante dentro del procedimiento. Esta revisión permitió identificar posibles puntos de ruido, especialmente en algunas imágenes negativas, los cuales podrían afectar una futura etapa de clasificación. Por lo tanto, antes de generar el archivo CSV, fue necesario confirmar que las imágenes segmentadas representaran correctamente las muestras originales.
- La generación de los archivos `CSV.csv` permitió almacenar las imágenes segmentadas como vectores fila, junto con sus respectivas etiquetas. Esta estructura resulta útil porque permite organizar todas las muestras dentro de un único archivo, donde cada fila representa una imagen y

la última columna indica si la muestra corresponde a una clase positiva o negativa.

- Se comprobó que los datos almacenados en el archivo CSV eran correctos. Al reconstruir las imágenes a partir de las filas del archivo y compararlas con las imágenes segmentadas originales, se verificó visualmente que la información fue guardada adecuadamente. De esta forma, se confirmó que el archivo CSV generado puede ser utilizado como base para una etapa posterior de análisis o clasificación.
- El proceso de extracción adaptativa de características permitió transformar cada imagen segmentada en un vector numérico compuesto por métricas geométricas, poblacionales y de forma. Esto permitió reducir significativamente la cantidad de información utilizada por los clasificadores, pasando de trabajar con los 16384 píxeles originales a únicamente 17 características representativas por imagen.
- La normalización de los datos fue útil antes del entrenamiento de los modelos de clasificación, debido a que las características extraídas presentan diferentes rangos y escalas. Este preprocesamiento permitió mejorar la estabilidad y desempeño de los algoritmos evaluados.
- La comparación de distintos algoritmos de clasificación permitió analizar cómo diferentes modelos responden ante un mismo conjunto de características. Para ello se evaluaron modelos basados en árboles de decisión, métodos probabilísticos, máquinas de soporte vectorial y clasificadores basados en vecinos cercanos, utilizando métricas como *Accuracy*, *Precision*, *Recall* y *F1 Score*.
- A partir de los resultados obtenidos, se logró identificar el modelo con mejor desempeño general utilizando el criterio del mayor *F1 Score*. Esto permitió seleccionar automáticamente el clasificador más adecuado para el problema de detección de contaminación con granos de arroz. Dando como resultado que el mejor modelo fue SVM RBF C1.

VI. HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL UTILIZADAS

En este proyecto se utilizó Chat GPT para las siguientes tareas:

- Generar la Figura 1.
- Correcciones en problemas de compilación de los programas .py.
- Correcciones en el proceso de creación del informe en Latex.

Además se usó Claude para lo siguiente:

- Establecimiento de los parámetros más adecuados para la identificación de los granos de arroz.
- Creación del vector de características.
- Unificación del proceso de entrenamiento para los distintos modelos.

REFERENCIAS

- [1] OpenCV, "Open Source Computer Vision Library," 2026. Recuperado de: <https://github.com/opencv/opencv>. Accedido: 29 de abril de 2026.
- [2] J. Kittler y J. Illingworth, "Minimum Error Thresholding," *Pattern Recognition*, vol. 19, no. 1, pp. 41–47, 1986.
- [3] M. Yang, K. Kpalma y J. Ronsin, "A Survey of Shape Feature Extraction Techniques," 2008. Recuperado de: <https://hal.science/hal-00446037v1/file/ARS-Journal-SurveyPatternRecognition.pdf>. Accedido: 22 de mayo de 2026.
- [4] D. A. Khusna, A. Z. Arifin y D. Purwitasari, "Analysis of Shape Features for Lesion Classification in Dermoscopic Images," 2016. Recuperado de: https://pubs.aip.org/aip/acp/article-pdf/doi/10.1063/1.4958602/13252083/160009_1_online.pdf. Accedido: 22 de mayo de 2026.
- [5] OpenCV, "Contour Properties," 2026. Recuperado de: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_contours/py_contour_properties/py_contour_properties.html. Accedido: 22 de mayo de 2026.
- [6] S. Beucher y C. Lantuéjoul, "Use of Watersheds in Contour Detection," 1979. Recuperado de: <https://people.cmm.minesparis.psl.eu/users/beucher/publi/watershed.pdf>. Accedido: 22 de mayo de 2026.
- [7] OpenCV, "Image Segmentation with Watershed Algorithm," 2026. Recuperado de: https://docs.opencv.org/4.x/d3/db4/tutorial_py_watershed.html. Accedido: 22 de mayo de 2026.
- [8] C. Bishop, "Pattern Recognition and Machine Learning," Springer, 2006. Recuperado de: <https://link.springer.com/book/10.1007/978-0-387-45528-0>. Accedido: 22 de mayo de 2026.
- [9] L. Rokach y O. Maimon, "Decision Trees," 2005. Recuperado de: https://link.springer.com/chapter/10.1007/0-387-25465-X_9. Accedido: 22 de mayo de 2026.
- [10] H. Zhang, "The Optimality of Naive Bayes," 2004. Recuperado de: <https://www.cs.unb.ca/~hzhang/publications/FLAIRS04ZhangH.pdf>. Accedido: 22 de mayo de 2026.
- [11] C. Cortes y V. Vapnik, "Support-Vector Networks," 1995. Recuperado de: <https://link.springer.com/article/10.1007/BF00994018>. Accedido: 22 de mayo de 2026.
- [12] T. Cover y P. Hart, "Nearest Neighbor Pattern Classification," 1967. Recuperado de: <https://ieeexplore.ieee.org/document/1053964>. Accedido: 22 de mayo de 2026.
- [13] T. Fawcett, "An Introduction to ROC Analysis," 2006. Recuperado de: <https://people.inf.elte.hu/kiss/13dwhdm/roc.pdf>. Accedido: 22 de mayo de 2026.
- [14] D. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," 2011. Recuperado de: https://bioinfpublication.org/files/articles/2_1_1_JMCS.pdf. Accedido: 22 de mayo de 2026.